Prowadzący zajęcia: Artur Samojluk

Inżynieria promptów i MVC

Od precyzyjnego polecenia do działającego MVP projektu w wersji 1.0.

Cel ćwiczenia

Nauczycie się formułować, sprawdzać i poprawiać prompty do kodowania. Uporządkujecie aplikację według prostego MVC i uruchomicie MVP 1.0, które realizuje wybrane funkcje oraz zapisuje dane w bazie.

Pracujcie w zespołach z poprzednich laboratoriów. Wykorzystajcie mapę z laba 02, technologie i aktualny kod z laba 03. W tym ćwiczeniu MVP oznacza minimalną użyteczną wersję produktu, a MVC - sposób podziału odpowiedzialności w kodzie.

1. Budujcie prompt jak konkretne zlecenie

Skuteczne polecenie opisuje rezultat i sposób jego sprawdzenia. Rola „programisty” może określić sposób pracy, ale potrzebne są również wymagania, aktualne pliki i kryteria poprawności. Stosujcie poniższe techniki [1].

Technika	Jak zastosować ją w swoim projekcie
Cel i kontekst	Podaj cel funkcji, wybrane technologie, aktualny stan oraz istotne pliki.
Ograniczony zakres	Zleć jedną zmianę; wskaż elementy, które mają zachować dotychczasowe działanie.
Uporządkowane instrukcje	Oddziel: Kontekst, Zadanie, Ograniczenia, Wynik i Sprawdzenie.
Przykład wejścia i wyniku	Dodaj poprawne dane oraz przypadek błędny z oczekiwanym komunikatem.
Praca etapami	Najpierw uzgodnij plan, potem wykonaj szkielet i pojedyncze funkcje.
Iteracja z dowodami	Po uruchomieniu opisz rozbieżność, kroki odtworzenia i komunikat błędu.

2. Porównujcie ogólne i precyzyjne polecenia

Ogólne: „Zrób aplikację do rezerwacji sal w MVC”. Nie określa zakresu, reguł ani sposobu odbioru.

Precyzyjne: „W obecnym SalaSpot dodaj F02 - zapis rezerwacji. Kontroler obsługuje formularz, model sprawdza termin i zapisuje rekord. Dla zajętej sali pokaż komunikat. Zwróć zmienione pliki oraz kroki sprawdzenia”.

Zapisujcie użyty czat, datę, treść promptu i rzeczywisty wynik. Przy nowej rozmowie przekażcie aktualny stan projektu. Poprawiajcie brakujący kontekst lub warunki, zamiast dopisywać ogólne żądanie „zrób lepiej”.

Prosty podział MVC

Zastosujcie trzy czytelne role. Kontroler koordynuje obsługę żądania; model zawiera dane i reguły, a widok odpowiada za prezentację [2].

3. Przypiszcie odpowiedzialność plikom

Element	Odpowiedzialność	Przykład SalaSpot
Model M	Odczyt i zapis danych oraz reguły biznesowe.	Sprawdzenie kolizji terminu i zapis rezerwacji.
Widok V	Układ strony, formularze, dane i komunikaty.	W02 - wybór sali oraz początku i końca terminu.
Kontroler C	Odbiór żądania, wywołanie modelu i wybór odpowiedzi.	Odczyt formularza, wywołanie modelu, pokazanie wyniku.

Przykładowa organizacja dla Python + Flask + SQLite:

Plik lub katalog	Rola w prostym projekcie
app.py	Tworzy aplikację i rejestruje kontroler.
controllers/rezerwacje.py	Kontroler z trasami GET i POST, np. Blueprint we Flask.
models/rezerwacja.py	Funkcje pobierania, tworzenia i anulowania rezerwacji.
templates/rezerwacje/	Widoki HTML listy i formularza.
db.py i schema.sql	Połączenie z SQLite i struktura bazy używana przez model.

We Flask funkcje obsługujące trasy są nazywane „view functions”. W przyjętym tu podziale pełnią rolę kontrolera, a widokami są szablony HTML [3]. Ten podział jest konwencją ćwiczenia. Przy innym stosie zachowajcie te same odpowiedzialności.

P1 Plan MVP i mapa MVC - szablon do uzupełnienia

Pomóż nam zaplanować MVP 1.0 projektu [tytuł]. Cel użytkownika: [cel]. Stos: [technologie]. Zakres: [3-5 funkcji F01...]. Widoki i dane: [mapa z laba 02]. Aktualny stan i pliki: [opis i kod]. Przygotuj plan wykorzystania istniejącego kodu z podziałem na model, widok i prosty kontroler. Dla każdej funkcji wskaż pliki MVC oraz warunek poprawności. Wypisz braki i założenia. Zachowaj uzgodniony zakres MVP. Wynik: mapa plików i kolejność małych zmian. Na tym etapie przedstaw plan.

Przykład uzupełnienia: SalaSpot; cel - znaleźć salę i zarezerwować termin; F01 dostępność, F02 rezerwacja, F03 anulowanie. Zaakceptujcie plan dopiero po porównaniu go z własną mapą projektu.

Prompty do wykonania aplikacji

Poniższe szablony dostosujcie do własnego projektu. Zastąpcie nawiasy konkretnymi informacjami i dołączcie aktualne pliki.

4. Wykonajcie szkielet MVC i pojedyncze funkcje

P2 Działający szkielet lub uporządkowanie kodu MVC

Wykonaj pierwszy etap zaakceptowanego planu [plan z P1] dla [tytuł]. Technologie: [stos]. Aktualne pliki: [kod]. Wyodrębnij model, widoki i kontroler. Dostosuj istniejące połączenia między plikami i zachowaj działające funkcje oraz dane. Kontroler ma odbierać żądanie, wywoływać model i wybierać widok lub przekierowanie. Przygotuj jedną działającą ścieżkę od adresu strony do danych i widoku. Zwróć drzewo katalogów, pełny kod zmienionych plików, zależności i polecenia uruchomienia dla [system]. Kryterium: aplikacja startuje i pokazuje dane z modelu.

Przykład zadania do P2: kontroler pobiera listę sal z modelu i przekazuje ją do W01. Uruchomcie tę ścieżkę, zanim przejdziecie do kolejnej funkcji.

P3 Jedna funkcja MVP z przykładami wyniku

W aktualnym kodzie [pliki] wykonaj wyłącznie [ID i opis funkcji]. Użytkownik: [rola]. Dane wejściowe: [pola]. Reguły modelu: [warunki]. Kontroler ma wywołać model i przekazać wynik do [widok]. Przykład poprawny: [wejście -> wynik]. Przykład błędny: [wejście -> komunikat]. Zapisuj dane w obecnej bazie; używaj zapytań parametryzowanych. Zachowaj wcześniejsze funkcje. Zwróć zmienione pliki i kroki sprawdzenia przypadku poprawnego, błędnego oraz zapisu po restarcie.

Przykład do P3 dla F02: ta sama sala jest zajęta 10:00-11:00. Nowe 10:30-11:30 ma zostać odrzucone, a 11:00-12:00 przyjęte. Sprawdzajcie datę i godzinę oraz aktywny status rezerwacji.

Przykład fragmentu prostego kontrolera we Flask

Fragment poglądowy dla wariantu z użytkownikiem demonstracyjnym nr 1. Zakłada zarejestrowany Blueprint bp, import render_template oraz funkcję modelu pobierz_rezerwacje. Pełny kod i inicjalizację przygotujcie w P2.

```
@bp.get("/rezerwacje")
def lista():
    rezerwacje = pobierz_rezerwacje(uzytkownik_id=1)
    return render_template("rezerwacje/lista.html",
                           rezerwacje=rezerwacje)
```

Prześledźcie: żądanie trafia do lista(), model pobiera dane, a szablon wyświetla wynik. Dla zapisu kontroler przekazuje dane do funkcji modelu, która sprawdza reguły i zapisuje rekord.

Uruchomienie i odbiór MVP 1.0

5. Poprawiajcie kod na podstawie rzeczywistych wyników

P4 Diagnoza i poprawka przed odbiorem - szablon

Sprawdź zgłoszoną niezgodność z MVP 1.0. Aktualne pliki: [kod]. Sposób uruchomienia i wersje narzędzi: [opis]. Kroki odtworzenia: [kroki]. Oczekiwane działanie: [wynik]. Faktyczny wynik i komunikat: [dowody]. Ustal przyczynę i zaproponuj najmniejszą poprawkę zachowującą podział MVC. Zwróć zmienione pliki, krótkie wyjaśnienie i scenariusze ponownego sprawdzenia tej funkcji oraz wcześniejszych funkcji. Odróżnij testy wykonane od proponowanych; potwierdzaj uruchomienie tylko wtedy, gdy rzeczywiście je wykonano.

Przykład do P4: po restarcie znika rezerwacja zapisana w F02. Dołączcie polecenie startu, położenie pliku bazy i aktualny kod inicjalizacji. Jeżeli wszystko działa, zamiast zgłaszać błąd poproście o przegląd zgodności z mapą MVP.

6. Wykonajcie zadania w swoim projekcie

Zadanie	Co wykonujecie i zapisujecie
1 Doprecyzowanie	Napiszcie ogólny prompt, poprawcie go według sekcji 1 i użyjcie P1. Zapiszcie własny plan MVC.
2 Szkielet i funkcje	Wykonajcie P2, a potem P3 osobno dla każdej z 3-5 funkcji MVP. Po każdym kroku uruchomcie kod i zapiszcie commit.
3 Próba i poprawka	Wykonajcie rzeczywiste testy. Użyjcie P4 do błędu lub przeglądu. Zapiszcie poprawiony prompt i zmianę w wyniku.
4 Wersja 1.0	Uruchomcie projekt u obu osób według README. W przeglądarce przejdźcie główny proces. Oznaczcie sprawdzony stan jako v1.0.0.

7. Potwierdźcie gotowość wersji 1.0

- ☐ Działają wszystkie 3-5 funkcji MVP i pełna ścieżka użytkownika od widoku do zapisu danych.
- ☐ Model, widok i kontroler są rozdzielone; obie osoby potrafią wyjaśnić przepływ jednej funkcji.
- ☐ Dla każdej funkcji sprawdzono przypadek poprawny i błędny; zapisane dane pozostają po restarcie.
- ☐ README pozwala uruchomić projekt od początku; wyniki testów mają status działa / błąd / niewykonany.

Do oddania i termin

Repozytorium z kodem 1.0 i README; docs/architektura-mvc.md z mapą plików; docs/prompty-mvc-v1.0.md z czterema uzupełnionymi szablonami, poprawką promptu i wynikami; docs/testy-v1.0.md z rzeczywistymi testami. W LaboChat.com zaktualizujcie zadania i linki. Każda osoba wnosi własny commit. Termin: do kolejnych zajęć.

Materiały pomocnicze (sprawdzone 03.11.2026): [1] OpenAI - projektowanie promptów; [2] MDN - MVC; [3] Flask - obsługa żądań.